QATIP Intermediate AWS Lab 05
Validations and Role-Based Access in AWS

Overview

This lab introduces Terraform validation mechanisms (preconditions and postconditions)

Before you begin

1. Ensure you have completed Lab0 before attempting this lab.
2. In the IDE terminal pane, enter the following command…

   cd c:\aws-tf-int\labs\05

3. This shifts your current working directory to labs/05. Ensure all commands are executed in this directory
4. Close any open files and use the Explorer pane to navigate to and open the labs/05 folder.

Phase 1 – EC2 Instance Type Precondition Check

Purpose

- Demonstrate how Terraform can enforce guardrails to prevent undesired configurations.

Steps Summary

- Modify instance type to m5.large in terraform.tfvars.
- Run terraform plan — observe failure due to precondition.
- Revert to t2.small — observe planning success.

©QA 2026

Rationale

- Preconditions help enforce organizational standards before provisioning happens. In this case, instance types not beginning with t2 are disallowed.

Steps

1. Update **terraform.tfvars** to attempt to provision an EC2 instance of type 'm5.large'.
2. Run;

   terraform init
   terraform plan

3. Terraform planning fails with a validation error (precondition). The precondition checks that the variable **var.instance_type** begins with "t2"
4. Update terraform.tfvars, reverting the instance type to **“t2.small”**
5. Run;

   terraform plan

6. The planning is successful as the variable **var.instance_type** now meets the precondition.

Phase 2 – Bucket Name Validations (Precondition and Postcondition)

Purpose

- Explore the difference between validating input variables (precondition) and verifying actual outcomes (postcondition) after logic is applied.

Steps Summary

- Attempt to create an S3 bucket that violates a precondition.
- Modify the bucket parameters to meet the precondition.
- Introduce a logic error to invoke a postcondition failure.
- Undo the error to meet the postcondition.

© QA 2026

- Apply the deployment and verify resources are created.
- Clean up by destroying resources.

Rationale

Preconditions catch bad input early; postconditions ensure that no unexpected transformations occur during deployment logic. This phase simulates bugs or misconfigurations that can pass initial validation but result in unintended infrastructure. Postconditions help catch these issues.

Steps

1. Update **terraform.tfvars**, changing the **bucket_name** variable value from "lab-bucket-xxxxxx" to "my-bucket-xxxxxx"
2. Run **terraform plan**
3. Terraform planning fails with a validation error (precondition at lines 36-39). The precondition checks that the variable **var.bucket_name** contains "lab-bucket"
4. Undo the changes to terraform.tfvars, reverting the bucket_name variable back to "lab-bucket-xxxx"
5. Run **terraform plan**
6. The planning should be successful as the preconditions are now met
7. You will now introduce a logic error that will be captured by a postcondition check
8. Modify line **28** of main.tf, replacing the 2nd instance of "**lab-bucket**" with "**test-bucket**". This is to simulate a logic error whereby the bucket name is changed from the value supplied as a variable (which passes the precondition check) resulting in the actual bucket name being something other than intended. The postcondition check validates the actual name that would be given to the bucket once all terraform logic is applied.
9. Run **terraform plan**
10. Terraform fails with a validation error (postcondition at lines 41-45).
11. Revert the changes to line **28**, re-aligning the actual bucket name back to the name derived from the variable.

12. Run **terraform plan**
13. Verify successful deployment with **terraform apply --auto-approve**
14. Switch to UI to verify EC2 instance and S3 bucket are created
15. Clean up with **terraform destroy --auto-approve**

What You Demonstrated

Terraform Preconditions and Postconditions:
You implemented both:

- **Preconditions** to confirm user input and enforce naming/instance standards.
- **Postconditions** to catch unexpected results or logic errors that could lead to misconfigured infrastructure.

***** Congratulations, you have completed this lab *****

©QA 2026

Copyright
© 2026 QA Michael Coulling-Green. All rights reserved. These lab materials are provided as part of an authorised training course. Course attendees may reuse these materials for their own personal learning and reference outside of the training session. Unauthorised copying, redistribution, publication, or use of these materials to deliver training is prohibited without prior written permission from the author.

© QA 2026